

LABORATOR 12

SISTEME ÎN TIMP REAL CU PROCESARE PARALELĂ. UTILIZAREA LIMBAJULUI VHDL PENTRU PROIECTAREA SISTEMELOR ÎN TIMP REAL

1. Obiectivul laboratorului

O metodă de a descrie circuitele hardware pentru implementarea acestora pe FPGA este utilizarea limbajului de descriere hardware VHDL. Limbajul VHDL permite atât descrierea unor sisteme la nivel de circuite logice componente dar și descrieri la nivel de procese ale unor sisteme cu un grad mai mare de complexitate. În acest laborator se va studia pentru început proiectarea unor sisteme aritmetice care permit operarea în paralel. Astfel, fiecare operație în parte se poate considera ca fiind un task ale cărui intrări reprezintă operanzii. Răspunsul (rezultatul) va fi generat în timp real, simultan pentru două sau mai multe operații care se vor efectua în paralel.

Implementarea se va face pe circuite FPGA , din seria Spartan 3, utilizând mediul de programare, sinteză și implementare ActiveHDL disponibil în laborator. Atragem atenția că la implementarea acestor procese paralele vorbim de un paralelism real și nu unul bazat pe concurența între procese ca până acum. Prin urmare, răspunsul care vine de la un sistem implementat în FPGA este unul mult mai rapid decât în cazul implementărilor pe sisteme cu un singur procesor care le-am utilizat până acum.

Etapele legate de crearea proiectelor pentru VHDL în Active HDL și de implementarea acestora pe FPGA au fost tratate la laboratoarele altor discipline. Pentru reamintirea etapelor parcurse în lucrul cu ActiveHDL și limbajul VHDL în directorul unde se află laboratoarele există o anexă a cărei obiectiv este prezentarea acestora.

Obiectivul propus în această lucrare este acela de a proiecta o unitate aritmetică paralelă, pentru operanzi pe 4 biți. Acesta trebuie să genereze rezultatul unei scăderi și a unei adunări în utilizând două unități de calcul care lucrează simultan. Proiectarea celor două unități se va face pornind de la structurile de bază, bazate pe porți logice.

2. Considerente teoretice

Descrierea în VHDL a unui sistem la nivel structural presupune specificarea componentelor din care sistemul este constituit și modul în care acestea sunt interconectate. Componentele pot fi la nivel de porți logice elementare dar pot fi și circuite mai complexe, descrise ulterior în alte porțiuni ale proiectului. Conectările între componente se fac prin intermediul instrucțiunilor de atribuire și prin hărți de porturi în cadrul apelului unei componente.

2.1. Atribuirea semnalelor

Atribuirea semnalelor se face în limbajul VHDL prin simbolul `<=` de forma următoare:

Semnal_destinație <= *Semnal_sursă*;

Semnificația instrucțiunii de mai sus este următoarea: *Semnal_destinație* este conectat la *Semnal_sursă*. Semnalul destinație este întotdeauna unul singur în timp ce semnalul sursă poate fi un singur semnal sau mai multe aflate într-o anumită relație (logică, aritmetică). Deci se poate vorbi de o conectare de tipul rezultat, și anume: rezultatul unei operații dintre mai multe semnale este conectat la semnalul destinație.

O condiție esențială la atribuire este aceea ca dimensiunea semnalelor din ambele părți ale operatorului să fie aceeași.

2.2. Operații logice

Fiecare instrucțiune din VHDL reprezintă de fapt descrierea unui circuit. Pentru circuitele porți logice elementare există cuvinte cheie alocate în VHDL. Astfel:

- SI logic se descrie utilizând cuvântul cheie *and*.
Exemplu: *t1 <= a and b;*
- SAU logic se descrie utilizând cuvântul cheie *or*.
Exemplu: *co <= t1 or t2;*
- SAU EXCLUSIV logic se descrie utilizând cuvântul cheie *xor*.
Exemplu: *stemp <= a xor b;*
- INVERSAREA sau CC1 se reprezintă utilizând cuvântul cheie *not*.
Exemplu: *b_int <= not b;*

Toate expresiile care în exemplele de mai sus au 2 operanzi pot avea mai mulți pentru a descrie porți logice elementare cu mai mult de 2 intrări. De asemenea se pot declara și expresii logice combinate pentru a desemna mai multe porți logice interconectate.

Exemplu: *s <= not (stemp xor ci);*

2.3. Declararea componentelor

La proiectarea unor sisteme mai complexe se folosește foarte des metoda bottom – up. Aceasta presupune inițial proiectarea circuitelor componente, de complexitate mai mică, urmând apoi integrarea acestora în sistemul complex. În cazul în care circuitul care se dorește inclus într-un sistem este declarat în altă parte a proiectului, este într-o librărie a limbajului VHDL sau într-un set de primitive, se utilizează declarația de componente. Aceasta se face prin cuvântul cheie ***component***.

Structura unei declarații de componente este prezentată mai jos:

```
component nume_circuit is  
    port(  
        ... declarații porturi...  
    );  
    generic(  
        ...declarații parametrii...  
    );  
end component;
```

nume_circuit : reprezintă numele circuitului care se dorește integrat așa cum acesta a fost declarat în entitatea sa.

Declarațiile de porturi și de parametrii trebuie să urmărească exact denumirile care au fost utilizate în entitate.

Declarația unei componente se face după ce a fost precizat numele arhitecturii din care componenta va face parte, dar înainte de a începe corpul propriu zis al arhitecturii (înainte de „begin”).

Apelarea componentei se face în corpul arhitecturii. O componentă se apelează utilizând următorul format:

nume_circuit_local : *nume_circuit* **port map** (...asocieri de porturi...) **generic map** (...asocieri de parametrii...);

nume_circuit_local : reprezintă numele care îl va avea circuitul în sistemul în care este integrat. Pentru un sistem care are mai multe circuite va trebui să se utilizeze denumiri diferite la fiecare. Singura excepție este atunci când se utilizează generatorul ciclic de circuite.

nume_circuit: reprezintă numele circuitului integrat, așa cum acesta a fost declarat în entitatea proprie.

Asocierile de porturi și asocierile de parametrii se fac prin listarea semnalelor care se doresc conectate la porturile respective. Dimensiunea și tipul semnalelor conectate la porturi trebuie să fie aceleași cu cele declarate în entitatea componentei.

În cazul în care se dorește integrarea într-un sistem a mai multor circuite de același tip se recomandă utilizarea generatorului ciclic de circuite. Declararea acestuia, tot în interiorul arhitecturii se face în felul următor:

nume_grup_circuite : **for** *i* **in** *valoare_inicială* **to** *valoare_finală* **generate**
...apeluri componente...

end generate;

nume_grup_circuite: reprezintă numele grupului de circuite așa cum acesta va fi recunoscut în sistem. Dacă sunt declarate mai multe grupuri de circuite, fiecare va trebui să aibă propriul său nume, excepție făcând cazul în care un grup de circuite este declarat la rândul lui într-un generator ciclic de circuite.

Contorul de circuite *i* poate lua orice valoare inițială și finală (numere întregi) și poate fi utilizat și ca index de semnal în cazul utilizării unor vectori de semnale în apelul componentelor din interiorul generatorului.

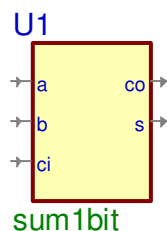
Alături de componente se pot efectua, în generatorul ciclic, operații de atribuire de semnale sau chiar alte generatoare ciclice de circuite.

3. Desfășurarea lucrării

1. Proiectarea circuitului de adunare în VHDL.

După crearea unui proiect în ActiveHDL (etape prezentate în anexa care există în același director cu platforma de laborator), pentru proiectarea circuitului de adunare se vor urma pașii:

- se va crea un fișier nou, VHDL, utilizând wizard-ul, denumit sum1bit.vhd. Aici se va descrie sumatorul complet pe un bit care va intra în componența atât a circuitului de adunare cât și a celui de scădere. Structura sumatorului pe un bit, la nivel de semnale de intrare și ieșire este prezentată în figura de mai jos.



Circuitul are 3 intrări pe un bit (a,b,ci) și două ieșiri pe un bit (co,s).
 Declarația acestuia în VHDL, după construirea fișierului vhd, va fi următoarea:

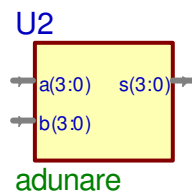
```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity sum1bit is
    port(
        a : in STD_LOGIC;
        b : in STD_LOGIC;
        ci : in STD_LOGIC;
        s : out STD_LOGIC;
        co : out STD_LOGIC
    );
end sum1bit;
```

architecture sum1bit_a of sum1bit is

```
signal stemp,t1,t2 : std_logic;
begin
    stemp <= a xor b;
    t1 <= a and b;
    s <= stemp xor ci;
    t2 <= stemp and ci;
    co <= t1 or t2;
end sum1bit_a;
```

- b. se va crea un fișier denumit adunare.vhd. Acesta va reprezenta circuitul de adunare a două numere pe patru biți fiecare. Structura sa este reprezentată în figura de mai jos.



Circuitul are două intrări pe 4 biți fiecare și o ieșire pe 4 biți. Structura sa internă, după crearea fișierului, trebuie să fie următoarea.

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity adunare is
    port(
        a : in STD_LOGIC_VECTOR(3 downto 0);
```

```

        b : in STD_LOGIC_VECTOR(3 downto 0);
        s : out STD_LOGIC_VECTOR(3 downto 0)
    );
end adunare;

architecture adunare_a of adunare is
    signal trans : std_logic_vector(4 downto 0);
    component sum1bit is
        port(
            a : in STD_LOGIC;
            b : in STD_LOGIC;
            ci : in STD_LOGIC;
            s : out STD_LOGIC;
            co : out STD_LOGIC
        );
    end component;
begin

    sumatoare: for i in 0 to 3 generate
        suma : sum1bit port map (a(i),b(i),trans(i),s(i),trans(i+1));
    end generate;

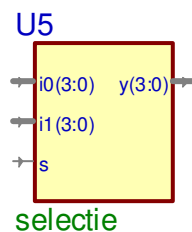
    trans(0) <= '0';

end adunare_a;

```

2. Proiectarea circuitului de scădere

Pentru circuitul de scădere este necesar proiectarea unui circuit de selecție pentru două numere pe 4 biți fiecare. Circuitul de selecție va fi descris într-un fișier nou creat, denumit selectie.vhd, și are următoarea structură:



Circuitul are 2 intrări pe 4 biți fiecare, denumite aici i0 și i1 și o intrare de selecție s pe un bit. De asemenea, circuitul are o ieșire pe 4 biți denumită y. Descrierea circuitului este prezentată în listingul de mai jos.

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;

```

```

entity selectie is
    port(
        s : in STD_LOGIC;
        i0 : in STD_LOGIC_VECTOR(3 downto 0);
        i1 : in STD_LOGIC_VECTOR(3 downto 0);
        y : out STD_LOGIC_VECTOR(3 downto 0)
    );
end selectie;

```

```

architecture selectie_a of selectie is
begin

```

```

    with s select
    y <= i0 when '0',
    i1 when '1',
    "0000" when others;

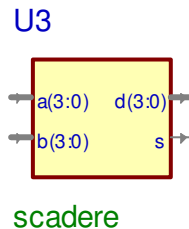
```

```

end selectie_a;

```

Structura circuitului de scădere, creat în fișierul scădere.vhd, este prezentată în figura de mai jos.



Circuitul are două intrări pe 4 biți fiecare (a și b), o ieșire pe 4 biți (d) și un bit de ieșire pentru semn (s).

Structura circuitului este declarată în listingul de mai jos.

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;

```

```

entity scadere is
    port(
        a : in STD_LOGIC_VECTOR(3 downto 0);
        b : in STD_LOGIC_VECTOR(3 downto 0);
        d : out STD_LOGIC_VECTOR(3 downto 0);
        s : out STD_LOGIC
    );
end scadere;

```

```

architecture scadere_a of scadere is
    signal a_int, b_int, s_int : std_logic_vector(4 downto 0);

```

```

signal s_intnot, s_int2 : std_logic_vector(3 downto 0);
signal trans : std_logic_vector(5 downto 0);
signal trans2 : std_logic_vector(4 downto 0);
component sum1bit is
    port(
        a : in STD_LOGIC;
        b : in STD_LOGIC;
        ci : in STD_LOGIC;
        s : out STD_LOGIC;
        co : out STD_LOGIC
    );
end component;
component selectie is
    port(
        s : in STD_LOGIC;
        i0 : in STD_LOGIC_VECTOR(3 downto 0);
        i1 : in STD_LOGIC_VECTOR(3 downto 0);
        y : out STD_LOGIC_VECTOR(3 downto 0)
    );
end component;
begin

    sumatoare : for i in 0 to 4 generate
        suma : sum1bit port map(a_int(i),b_int(i),trans(i),s_int(i),trans(i+1));
    end generate;

    sumatoare2 : for i in 0 to 3 generate
        suma2 : sum1bit port map('0',s_intnot(i),trans2(i),s_int2(i),trans2(i+1));
    end generate;

    se : selectie port map(s_int(4),s_int(3 downto 0),s_int2(3 downto 0),d);

    trans(0) <= '1';
    trans2(0) <= '1';
    a_int(3 downto 0) <= a;
    a_int(4) <= '0';
    b_int(3 downto 0) <= not b;
    b_int(4) <= '1';
    s <= s_int(4);
    s_intnot <= not s_int(3 downto 0);

end scadere_a;

```

3. Descrierea întregului sistem:

Pentru a descrie întreg sistemul este necesară și proiectarea unui circuit decodor 7 segmente care permite afișarea cifrelor pe display-ul cu LED-uri.

Decodorul 7 segmente se va descrie într-un fișier denumit decodor.vhd. Structura acestuia este prezentată în figura de mai jos.



Circuitul are o intrare pe 4 biți (digital) și o ieșire pe 7 biți (led).
Descrierea acestuia în VHDL este următoarea:

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;

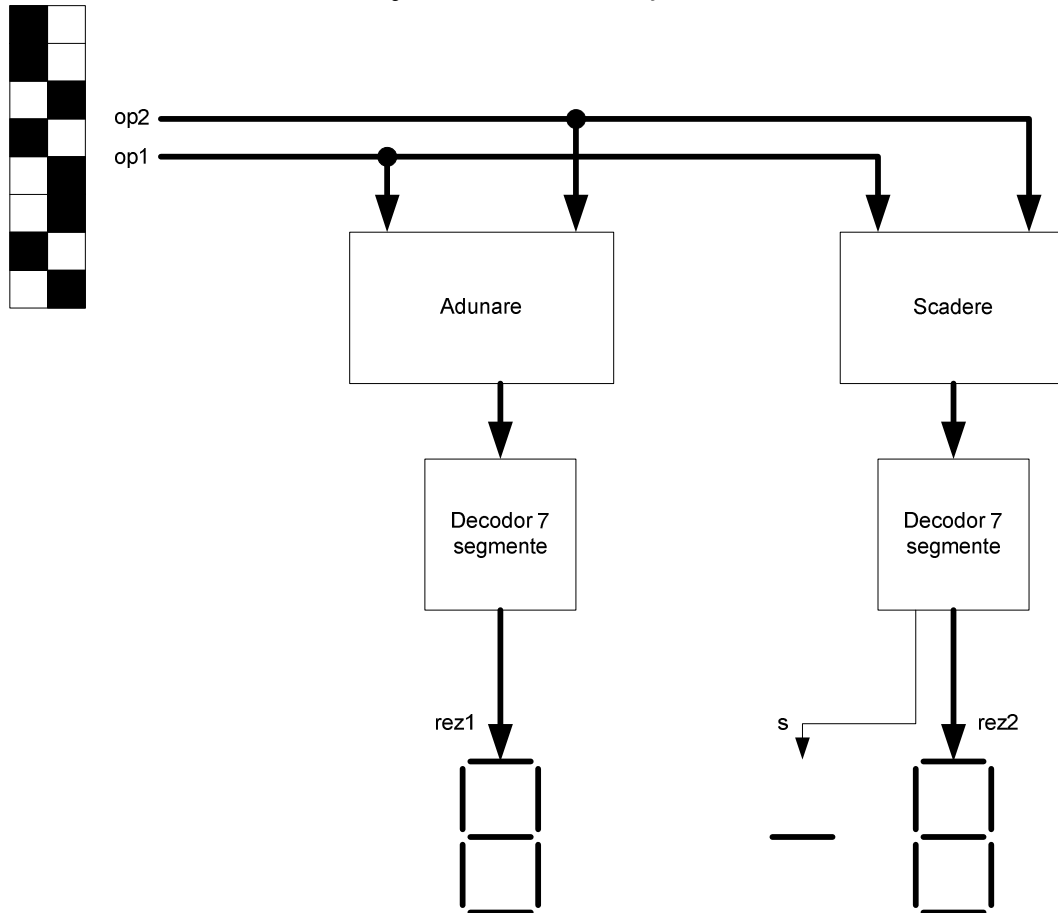
entity decodor is
    port(
        digital : in STD_LOGIC_VECTOR(3 downto 0);
        led : out STD_LOGIC_VECTOR(6 downto 0)
    );
end decodor;

architecture decodor_a of decodor is
    begin

        with digital select
            led <= "0111111" when "0000",
                  "0000110" when "0001",
                  "1011011" when "0010",
                  "1001111" when "0011",
                  "1100110" when "0100",
                  "1101101" when "0101",
                  "1111101" when "0110",
                  "0000111" when "0111",
                  "1111111" when "1000",
                  "1101111" when "1001",
                  "1110111" when "1010",
                  "1111100" when "1011",
                  "0111001" when "1100",
                  "1011110" when "1101",
                  "1111001" when "1110",
                  "1110001" when "1111",
                  "0000000" when others;

    end decodor_a;
    
```

Structura sistemului care se dorește realizat este prezentată în figura de mai jos. Sistemul va avea două intrări (op1 și op2) pe patru biți fiecare, două ieșiri (rez1 și rez2) pe 7 biți și o ieșire (semn) pe un bit. Intrările vor fi generate de la comutatoarele (SW_DIP) de pe bareta de comutatoare a machetei cu Spartan 3 iar ieșirile vor fi conectate la afișoarele 7 segmente. Sistemul va fi descris în limbajul VHDL într-un fișier nou, intitulat sistem.vhd.



Descrierea în VHDL a sistemului este prezentată în listing-ul de mai jos:

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity sistem is
  port(
    op1 : in STD_LOGIC_VECTOR(3 downto 0);
    op2 : in STD_LOGIC_VECTOR(3 downto 0);
    rez1 : out STD_LOGIC_VECTOR(6 downto 0);
    rez2 : out STD_LOGIC_VECTOR(6 downto 0);
    semn : out STD_LOGIC
  );
```

end sistem;

architecture sistem_a of sistem is

signal rez_a, rez_s : std_logic_vector(3 downto 0);

component adunare is

port(

a : in STD_LOGIC_VECTOR(3 downto 0);

b : in STD_LOGIC_VECTOR(3 downto 0);

s : out STD_LOGIC_VECTOR(3 downto 0)

);

end component;

component scadere is

port(

a : in STD_LOGIC_VECTOR(3 downto 0);

b : in STD_LOGIC_VECTOR(3 downto 0);

d : out STD_LOGIC_VECTOR(3 downto 0);

s : out STD_LOGIC

);

end component;

component decodor is

port(

digital : in STD_LOGIC_VECTOR(3 downto 0);

led : out STD_LOGIC_VECTOR(6 downto 0)

);

end component;

begin

ad : adunare port map(op1,op2,rez_a);

sc : scadere port map(op1, op2,rez_s,semn);

dec1 : decodor port map(rez_a,rez1);

dec2 : decodor port map(rez_s,rez2);

end sistem_a;

Intrările și ieșirile din sistem se vor asocia cu pinii FPGA care sunt conectați la grupul de comutatoare (SW_DIPx) sau la afișoarele de led-uri 7 segmente (DIGxSEGy). Valorile pinilor sunt date în anexa asociată cu acest laborator, în ultima parte alocată prezentării machetei.

Cerințe:

1. Să se creeze proiectul și să se construiască toate fișierele componente ale acestuia.
2. Să se deseneze schema hardware a sumatorului pe un bit și apoi a fiecărui circuit aritmetic în parte, pornind de la reprezentarea în VHDL cu declarația „component”.
3. Să se implementeze și să se simuleze sistemul cu cele două operații aritmetice.

4. Să se realizeze un nou sistem care are pe post de task-uri 2 operații logice pentru două numere pe 4 biți: SI logic și SAU logic.